

Frontend Engineering — Revised Edition

Phases 5 and 6: HTML, CSS, Tailwind & shadcn/ui, JavaScript, and TypeScript fundamentals, then a modern React + TypeScript application layer — forms, data fetching, state management, testing, and performance — wired up to the real Spring Boot API you built in Part A.

At a Glance	
Phases	5–6
Topics	15
Estimated Duration	8–10 weeks
Difficulty	Beginner → Intermediate
Suggested Pace	8–12 hrs/week · 2–3 topics/week (part-time default)
Projects Included	2–4 detailed hands-on projects per topic, step-by-step
Documentation Sources	2+ official/primary sources per topic

What Changed in This Revision

This edition implements a full curriculum review against professional 2026 frontend practice. **[NEW]** marks added topics, **[UPDATED]** marks deepened, reordered, or replaced ones. Unmarked topics are carried over from the original roadmap unchanged.

Replaced. Bootstrap / Tailwind CSS topic becomes Tailwind CSS & shadcn/ui — Bootstrap's JS-bundled component model is increasingly legacy in new React work; shadcn/ui (Radix primitives + Tailwind) is the pattern dominating new component libraries and connects directly into the Reusable Component Library project in Phase 6.

Deepened. CSS topic adds container queries and logical properties — the modern answer to component-level responsiveness, since a component rarely knows its own viewport.

Deepened. JavaScript (ES6+) topic adds the event loop (microtasks vs macrotasks) — the concept behind most confusing async bugs students later hit in React.

Deepened. React + TypeScript topic adds useReducer, custom hooks, error boundaries, and focus management — the last of which is accessibility-in-an-SPA, not just accessibility-in-static-HTML.

Added. Component Architecture & Composition Patterns — compound components, controlled vs uncontrolled, prop drilling vs lifting state, taught explicitly instead of hoped-for.

Added. Forms with React Hook Form + Zod — hand-rolled form state is exactly the repetitive work the ecosystem solved years ago.

Replaced. Axios & API Integration becomes Axios + TanStack Query — Axios stays for the client/interceptor layer, but the actual fetching/caching pattern is now React Query instead of manual useEffect.

Added. State Management: Context vs Zustand — separates server state (TanStack Query) from client/UI state (Zustand), and names when Context genuinely is the right tool vs. when it isn't.

Added. Testing: Vitest + React Testing Library + Playwright — a full frontend curriculum was shipping zero tests; this closes that gap right after components are built, not retrofitted later.

Added. Performance: code-splitting, Suspense, Lighthouse & Core Web Vitals — the same 'measure before optimizing' instinct already used well in Part A's database phase, applied to the frontend.

Added. Next.js Primer — SSR, file-based routing, and Server Components in one guided rebuild of a single page, so the SPA-only mental model isn't the only one a student leaves with. Deliberately scoped as a primer, not a full framework migration.

Updated. Final project checklists for both phases now include forms via RHF+Zod, TanStack Query data fetching, component + E2E tests, one code-split route, and a recorded Lighthouse/Core Web Vitals pass.

Updated. At-a-Glance figures, topic count, and progress tracker revised to match the new topic totals.

Topics Covered

PHASE 5 — Frontend Foundations

HTML · CSS (+ Container Queries) · Tailwind CSS & shadcn/ui (replacing Bootstrap) · JavaScript ES6+ (+ Event Loop) · TypeScript

PHASE 6 — Frontend Application Layer

React + TypeScript (+ useReducer, custom hooks, error boundaries) · Component Architecture & Composition Patterns (NEW) · Forms with React Hook Form + Zod (NEW) · Axios + TanStack Query (updated) · State Management: Context vs Zustand (NEW) · Types of APIs You'll Work With · Testing: Vitest, RTL & Playwright (NEW) · Performance: Code-Splitting & Core Web Vitals (NEW) · Frontend Dev Tools (+ Storybook) · Next.js Primer (NEW)

Next: with a working, tested, performance-checked React frontend talking to a real backend, move on to Part C — Deployment, Cloud & Integrations.

Frontend Foundations

HTML · CSS (+ Container Queries) · Tailwind & shadcn/ui · JavaScript (ES6+) · TypeScript

Before React, get fluent in the raw languages the browser actually runs. This is what lets you debug a layout bug or a broken event handler without guessing — and it's what separates someone who can use React from someone who understands it.

HTML

The structural skeleton of every web page — semantic, accessible markup that means something, not just divs everywhere.

Subtopics: *semantic HTML5 elements · forms & validation attributes · accessibility (ARIA, alt text) · meta tags & SEO basics*

1. Semantic Portfolio Page — structure before style

Use header, nav, main, section, article, footer correctly → no CSS yet — structure only → validate with the W3C HTML validator → view the outline in browser dev tools

2. Accessible Contact Form — forms that work for everyone

Add a proper label for every input → use the right input types (email, tel, etc.) → add required and pattern validation attributes → tab through it using only the keyboard

3. ARIA-Enhanced Widget — accessibility beyond plain HTML

Build a simple dropdown or accordion → add appropriate ARIA roles and aria-expanded → test it with a screen reader or accessibility dev tools → fix any issues the audit flags

Study Resources

- MDN: HTML Basics
- MDN: Learn HTML
- web.dev: Learn HTML

CSS [UPDATED]

Controlling layout and visual presentation — the box model, modern layout systems, and responsive design, now including component-level responsiveness.

Subtopics: *box model & positioning · Flexbox · Grid · responsive design/media queries · CSS variables & transitions · container queries (NEW) · logical properties (NEW)*

1. Responsive Navbar — Flexbox in a real component

Build a horizontal navbar with Flexbox → add a media query collapsing it to a mobile menu → add a smooth transition on open/close → test at 3 viewport widths

2. CSS Grid Photo Gallery — Grid for two-dimensional layout

Build a responsive grid with auto-fit/auto-fill → add a hover effect scaling each photo → make the gallery reflow from 4 to 2 to 1 columns

3. Container Query Card (NEW) — respond to the parent, not the viewport

Build a Card component with a title, image, and description → wrap it in container-type: inline-size → write container query breakpoints that switch the card from stacked to side-by-side → drop the same card into a narrow sidebar and a wide main column and confirm it adapts to each independently of viewport width

Study Resources

- MDN: CSS Reference

- CSS-Tricks: A Complete Guide to Flexbox
- MDN: CSS Container Queries

Tailwind CSS & shadcn/ui [UPDATED]

Utility-first styling plus the headless-component pattern (Radix primitives + Tailwind) that's now dominating new React component libraries — replacing Bootstrap, which is increasingly a legacy pattern in greenfield React work.

Subtopics: Tailwind utility classes & config · responsive breakpoint prefixes · shadcn/ui: copy-in components, not npm-installed · Radix primitives under the hood · theming with CSS variables

1. Portfolio Rebuild — Tailwind — utility-first styling

Rebuild your Phase 5 HTML portfolio using only Tailwind utility classes, no custom CSS file → use responsive prefixes (sm:/md:/lg:/) for breakpoints → note the resulting HTML verbosity

2. Custom Tailwind Config — go beyond the defaults

Extend tailwind.config with a custom color palette and font family → use your custom tokens in a component → confirm the build only includes classes actually used

3. shadcn/ui Component Pull (NEW) — the copy-in pattern

Run the shadcn/ui CLI to add a Dialog and a Select into your project → open the generated files and read them — this code now lives in your repo, not node_modules → customize the Dialog's styling directly → explain in a comment why 'copy the code in' is a different trust model than a normal npm install

4. Bootstrap-for-Comparison (kept, trimmed) — recognize the pattern in legacy codebases

Add Bootstrap via CDN to a throwaway page → drop in a navbar and a modal using only prebuilt classes/components → note how much JS behavior comes bundled in versus how much Tailwind+shadcn required you to wire yourself → write 2 bullet points on when you'd still choose Bootstrap (e.g. an internal admin tool with a tight deadline)

Study Resources

- Tailwind CSS Documentation
- shadcn/ui Documentation
- Radix UI Primitives

JavaScript (ES6+) [UPDATED]

The language that actually runs in the browser — modern syntax, the DOM, and asynchronous programming, now including the execution model behind the async bugs you'll hit in React.

Subtopics: let/const, arrow functions, destructuring · DOM manipulation & events · Promises & async/await · closures & modules · array methods (map/filter/reduce) · the event loop: microtasks vs macrotasks (NEW)

1. Vanilla JS To-Do List — DOM manipulation fundamentals

Render a list from a JS array → add an input + button to add new items → add click handlers to remove/complete items → pure in-memory state

2. Weather App — real async data fetching

Call a public weather API with fetch() → handle the loading state while waiting → handle the error state if the call fails → render the result once it resolves

3. Event Loop Prediction Drill (NEW) — build intuition before it costs you a debugging session

Write a script mixing console.log, setTimeout(fn, 0), and Promise.resolve().then() → predict the exact print order on paper first → run it and compare → repeat with a fetch() call mixed in → write one paragraph explaining why microtasks always drain before the next macrotask

Study Resources

- MDN: JavaScript Guide

- javascript.info: The Modern JavaScript Tutorial
- MDN: Using Promises
- MDN: In depth — microtasks and the JavaScript runtime environment

TypeScript

JavaScript with a type system layered on top — catching a whole category of bugs before the code ever runs.

Subtopics: basic types & interfaces · union/intersection types · type narrowing · generics in TypeScript

1. To-Do List → TypeScript — convert a JS project to TS

Add a Task interface (id, text, done) → type every function's parameters and return value → fix every type error the compiler reports

2. Discriminated Union Practice — type narrowing on a shared 'kind' field

Model 2-3 shape types sharing a kind field (circle/square) → write a function using a switch on kind → confirm TypeScript narrows correctly inside each case → add a new shape and let the compiler flag the missing switch case

3. Strict Mode Migration — turn on the strictest settings and fix everything

Enable strict: true in tsconfig.json → fix every new error → pay special attention to implicit any errors → confirm the project still builds cleanly

Study Resources

- TypeScript Official Handbook
- TypeScript: TSConfig Reference
- Total TypeScript: Beginner's Tutorial

Final project for this phase — ShopFlow: Static Storefront

- Product listing and detail pages in semantic HTML
- Fully responsive layout (Grid/Flexbox), styled with Tailwind; at least one component (e.g. Card) rebuilt to respond to a container query, not just the viewport
- NEW: one interactive element (e.g. the cart drawer or a filter panel) built with a shadcn/ui component instead of hand-rolled markup
- Cart UI in vanilla TypeScript — add/remove, running total, localStorage persistence
- Fetch product data from a local JSON file (stand-in for the real API)
- No framework yet — proves you can build a real UI with just HTML/CSS/TS

This is your frontend fundamentals proof-of-concept, the same way Phase 1's console cart proved your OOP. Phase 6 rebuilds this in React and wires it to your real Spring Boot API.

PHASE 6 OF 10

Frontend Application Layer

React · Component Architecture · Forms · TanStack Query · State Management · Testing · Performance · Next.js Primer

Now connect your frontend fundamentals to your real Spring Boot backend from Part A. Deployment for both the frontend and backend gets its own full track in Part C — this phase focuses on the application code, the ecosystem tools a real 2026 React codebase actually uses, and the tools you'll use to build it.

React + TypeScript [UPDATED]

A component-based UI library, typed with TypeScript, for building the interface your users actually interact with — now including the hooks and resilience patterns a real app needs beyond the basics.

Subtopics: JSX, props, state, hooks · useReducer for complex local state (NEW) · custom hooks (NEW) · React Router (protected routes) · error boundaries (NEW) · focus management in an SPA (NEW)

1. Auth Pages — login/register with route protection

Build a Login and Register form with controlled inputs → call your Phase 3 auth API on submit → store the returned JWT → build a ProtectedRoute wrapper redirecting unauthenticated users

2. Product Listing Page — fetch, filter, and display real data

Fetch products from your Spring Boot API → add category filter and search inputs → add a loading skeleton → handle the empty-results case gracefully

3. Custom Hook Extraction (NEW) — the primary React code-reuse mechanism

Take the fetch-and-loading-state logic you just wrote twice → extract it into a useFetch or useProducts custom hook → reuse it across 2 different pages → confirm both pages now share one source of truth for that logic

4. Error Boundary + Focus Management (NEW) — resilience and accessibility together

Wrap a route in an ErrorBoundary catching render-time errors → show a fallback UI instead of a blank screen → separately, add focus-trapping to your auth modal and confirm focus returns to the trigger button on close → confirm a screen reader announces the route change

Study Resources

- React Official Documentation
- React Router Documentation
- TypeScript: React Cheatsheet
- React: Error Boundaries

Component Architecture & Composition Patterns [NEW]

How to structure components so the codebase stays maintainable as it grows — named and taught explicitly instead of something students stumble into via one project.

Subtopics: compound components · controlled vs uncontrolled components · prop drilling vs lifting state up · children-as-function / render props · when to split a component

1. Compound Component: Tabs — build the pattern from scratch

Build a API using React Context internally → confirm the parent Tabs manages active-tab state while children just render → compare the ergonomics to a single Tabs component taking a giant props object

2. Controlled vs Uncontrolled Audit

Take 2 form inputs you built earlier as controlled components → rebuild one as uncontrolled using a ref → write 3 bullet points on when you'd actually choose uncontrolled (e.g. a file input, a large form validated only on submit)

3. Reusable Component Library — Button, Input, Modal, Table, Pagination

Extract a Button and Input component with typed props → extract a reusable Modal → extract a Table and Pagination component → use all five across at least 2 different pages

Study Resources

- React: Passing Data Deeply with Context
- patterns.dev: Compound Pattern
- React: Sharing State Between Components

Forms with React Hook Form + Zod [NEW]

Hand-rolled form state (a useState per field, manual validation logic) is exactly the repetitive, error-prone work the ecosystem solved years ago. This is what a real form-heavy PR looks like.

Subtopics: React Hook Form: register, handleSubmit, formState · Zod schema validation · resolver integration (zodResolver) · field-level vs form-level errors · async validation

1. Checkout Form — the standard RHF + Zod workflow

Define a Zod schema for a checkout form (name, email, address, card fields) → wire it to React Hook Form via zodResolver → render field-level error messages → confirm submission is blocked until the schema passes

2. Async Validation — check something against the server

Add an email field that checks availability against your Spring Boot API as the user types → debounce the check → show a pending/valid/invalid state per keystroke pause

3. Reusable Form Field Components — connect back to Component Architecture

Extract a typed FormField wrapper combining a label, input, and error message → wire it to RHF's register → use it across the checkout form and the auth forms from the React topic → confirm one component now handles every form's field rendering

Study Resources

- React Hook Form Documentation
- Zod Documentation
- React Hook Form: Zod Resolver

Axios + TanStack Query [UPDATED]

Axios stays for the client/interceptor layer — but the actual data-fetching and caching pattern is TanStack Query, not manual useEffect. Caching, background refetching, and request deduplication are the actual hard parts of data fetching, and hand-rolling them is the pattern the industry has moved away from.

Subtopics: Axios: centralized instance & interceptors · TanStack Query: queries & mutations (NEW) · cache invalidation & refetching (NEW) · optimistic updates (NEW) · JWT attach + auto-refresh on 401

1. Axios API Client Setup — one instance, used everywhere

Create a single axios.create() instance with baseURL and default headers → add a request interceptor attaching the JWT automatically → add a response interceptor handling 401 with auto token refresh

2. Migrate Product Fetching to TanStack Query (NEW) — replace manual useEffect

Take your Product Listing page's fetch-in-useEffect logic → replace it with useQuery, passing your Axios instance as the fetcher → delete the manual loading/error state — useQuery already gives you both → confirm a second visit to the page shows cached data instantly while revalidating in the background

3. Optimistic Cart Mutation (NEW) — instant UI feedback with a rollback path

Wire the 'add to cart' action through useMutation → update the cart UI optimistically before the server responds → configure onError to roll back the optimistic update → test it by throttling the network in dev tools and watching the optimistic state, then the real confirmation

Study Resources

- Axios Documentation
- Axios: Interceptors Guide
- TanStack Query Documentation
- TanStack Query: Optimistic Updates

State Management: Context vs Zustand [NEW]

Separating server state (TanStack Query's job) from client/UI state (Context or Zustand's job) is the distinction most junior React work gets wrong. Context is taught as a ceiling in most bootcamps — this topic names when it genuinely is the right tool and when it isn't.

Subtopics: server state vs client state — the core distinction · Context API: when it's the right size · Context's re-render cost at scale · Zustand: a minimal external store · choosing per-feature, not project-wide

1. Cart State in Context — the appropriately-sized use case

Build cart state in a Context provider → add quantity controls and a remove button per item → compute and display the running total → persist the cart across a page refresh

2. Reproduce Context's Re-render Problem, Then Fix with Zustand (NEW)

Add a rapidly-updating value (e.g. a live cursor position or search-input state) to your existing cart Context → observe every consumer re-rendering on every keystroke, even unrelated ones, using React DevTools' render highlighting → move just that fast-changing value into a Zustand store → confirm unrelated components stop re-rendering

3. Server State vs Client State Audit (NEW)

List every piece of state in your ShopFlow frontend so far → mark each as 'server state' (belongs in TanStack Query) or 'client state' (belongs in Context/Zustand/useState) → move any misplaced state to the right layer → write 2 bullet points on how you'd explain the distinction to a teammate

Study Resources

- React: Passing Data Deeply with Context
- Zustand Documentation
- TkDodo's blog: Practical React Query (server vs client state)

Types of APIs You'll Work With

Recognizing the different API styles you'll encounter on the job, beyond the REST/GraphQL you've already built.

Subtopics: REST (recap) · GraphQL (recap) · WebSocket/Server-Sent Events · Webhooks · gRPC & SOAP (awareness only)

1. Live Order Status Widget (WebSocket) — a persistent connection instead of polling

Connect to a WebSocket or SSE endpoint from React → subscribe to order-status update events → update the UI in real time → confirm no polling/interval requests are happening

2. Compare Polling vs WebSocket — feel the difference directly

Build the same live-status feature with setInterval polling first → rebuild with WebSocket/SSE → compare network tab traffic → write 3 bullet points on the trade-offs

Study Resources

- MDN: WebSockets API
- MDN: Server-Sent Events
- Spring: WebSocket Support Reference

Testing: Vitest, React Testing Library & Playwright [NEW]

A frontend curriculum that ships zero tests trains habits that don't survive contact with a real team's CI pipeline. This lands right after components are built, not retrofitted at the end.

Subtopics: Vitest basics · React Testing Library: querying by role, not implementation detail · mocking API calls in component tests · one Playwright E2E flow · what to test vs what not to

1. Component Test: the Reusable Button/Input Library

Write Vitest + RTL tests for your Button and Input components → query by role/label, not by class name or test-id where avoidable → test a disabled state and a click handler firing

2. Mocked API Component Test

Test the Product Listing page → mock the Axios/TanStack Query call so no real network request happens → assert the loading skeleton renders first, then the product list → assert the empty-results message renders when the mock returns []

3. One Playwright E2E Flow — login → add to cart → checkout

Write a single Playwright test driving a real browser against your running app → log in, add a product to cart, complete checkout → assert the order appears in Order History → run it in headed mode once to watch it, then headless in CI mode

Study Resources

- Vitest Documentation
- React Testing Library Documentation
- Playwright Documentation
- Kent C. Dodds: Common Testing Mistakes

Performance: Code-Splitting & Core Web Vitals [NEW]

The same 'measure before optimizing' instinct already used well in Part A's database-indexing phase, applied to the frontend — profile first, then fix the specific thing that's actually slow.

Subtopics: React.lazy & Suspense · route-based code-splitting · bundle-size awareness · Lighthouse & Core Web Vitals (LCP, CLS, INP) · when useMemo/useCallback actually help

1. Baseline Lighthouse Pass — measure before optimizing

Run a Lighthouse audit on your React app as it stands → record LCP, CLS, and INP scores → identify the single largest JS bundle chunk in the network tab

2. Code-Split One Route with React.lazy

Wrap your least-visited route (e.g. Order History) in React.lazy + Suspense → confirm its JS chunk now loads only when that route is visited → rerun Lighthouse and compare the initial bundle size before/after

3. useMemo/useCallback: Prove It Before You Reach For It

Find a component re-rendering unnecessarily using React DevTools Profiler → add useMemo/useCallback where the profiler actually shows a cost → remove it from one place where it was added 'just in case' and confirm no measurable difference → write one paragraph on why memoizing everything by default is itself a performance anti-pattern

Study Resources

- React: React.lazy Reference
- web.dev: Core Web Vitals
- Chrome DevTools: Lighthouse
- React DevTools Profiler

Frontend Dev Tools [UPDATED]

The tools you'll reach for constantly while building the above — reference, not a project list.

Subtopics: npm/pnpm/yarn · Vite: dev server, HMR, build config · ESLint + Prettier · browser DevTools (Network, Console, Application) · React Developer Tools · Postman/Insomnia · Storybook for isolated component development (NEW)

1. Dev Tools Fluency Pass

Inspect a failed API call in the Network tab and identify the exact response body → use the Application tab to inspect your stored JWT and cart persistence → profile a re-render with React DevTools

2. Storybook Setup (NEW) — develop components in isolation

Add Storybook to your project → write a story for your Button and Modal components covering their different states (default, disabled, loading) → confirm you can develop and visually check a component without running the full app

Study Resources

- Vite Documentation
- ESLint Documentation
- Chrome DevTools Documentation
- Storybook Documentation

Next.js Primer [NEW]

Enough exposure to SSR, file-based routing, and Server Components to see the alternative to the SPA-only model — deliberately scoped as a primer, one guided page rebuild, not a full framework migration.

Subtopics: file-based routing · Server Components vs Client Components · SSR/SSG basics · why this solves SPA SEO · when you'd reach for Next.js vs stay SPA

1. Rebuild One Page in Next.js — feel the model shift

Create a fresh Next.js app → rebuild just your Product Listing page using the App Router → fetch products in a Server Component instead of useQuery → compare: no loading skeleton needed, because the HTML arrives with data already in it

2. Client Component Boundary — mix both models on purpose

Add an 'add to cart' button that needs interactivity → mark it "use client" → confirm the rest of the page stays server-rendered → write 3 bullet points on how you'd decide the client/server boundary on a real page

3. SEO Comparison — view-source doesn't lie

View-source your original SPA's Product Listing page and note the empty → view-source the same page rebuilt in Next.js and note the fully-rendered HTML → explain in a comment why this matters for a crawler that doesn't execute JavaScript

Study Resources

- Next.js Documentation
- React: Server Components
- Next.js: App Router

Final project for this phase — ShopFlow: React Frontend, Production-Ready

- Rebuild the storefront in React + TypeScript, wired to the real Spring Boot API from Part A
- Centralized Axios client with interceptors for JWT attach + auto-refresh on 401
- NEW: product and cart data fetched via TanStack Query instead of manual useEffect, with an optimistic add-to-cart mutation
- NEW: checkout and auth forms built with React Hook Form + Zod
- NEW: server state (TanStack Query) and client state (Context/Zustand) kept deliberately separate
- Live order status using WebSocket/SSE instead of polling
- Reusable component library (Button, Input, Modal, Table, Pagination) used consistently, developed in Storybook
- Full auth flow: register, login, protected routes, logout
- NEW: one Vitest + React Testing Library component test suite and one Playwright E2E flow (login → add to cart → checkout)
- NEW: at least one route code-split with React.lazy, with a before/after Lighthouse Core Web Vitals comparison
- NEW: one page rebuilt in the Next.js primer, with a view-source SEO comparison against the SPA version

Deploying this frontend (and the Spring Boot backend behind it) to a live URL is covered in full in Part C — Deployment & Cloud, right after External APIs.

Part B — Frontend Engineering — Progress Tracker (Revised)

Check off each topic once you've read its concept sections, built its hands-on projects, and folded the result into ShopFlow where indicated.

PHASE 5 — Frontend Foundations

- HTML
- CSS [UPDATED]
- Tailwind CSS & shadcn/ui [UPDATED]
- JavaScript (ES6+) [UPDATED]
- TypeScript

PHASE 6 — Frontend Application Layer

- React + TypeScript [UPDATED]
- Component Architecture & Composition Patterns [NEW]
- Forms with React Hook Form + Zod [NEW]
- Axios + TanStack Query [UPDATED]
- State Management: Context vs Zustand [NEW]
- Types of APIs You'll Work With
- Testing: Vitest, React Testing Library & Playwright [NEW]
- Performance: Code-Splitting & Core Web Vitals [NEW]
- Frontend Dev Tools [UPDATED]
- Next.js Primer [NEW]